

ML-ENGINEER-AGENT

Abstract

The **Autonomous ML Engineer Agent** is an AI-powered AutoML system that allows users to upload a raw dataset and automatically build a complete machine learning pipeline. The system analyzes the dataset, detects the task type, performs exploratory data analysis, handles missing values, preprocesses numerical and categorical features, trains multiple machine learning models, evaluates performance, selects the best model, generates an explainable report, and prepares a deployable prediction API.

The purpose of this project is to reduce the manual effort required in building machine learning models while still keeping the process transparent, explainable, and production-ready. Unlike a simple AutoML script, this system is designed as a modular multi-agent platform where each agent is responsible for one stage of the ML lifecycle.

Overall Workflow

```
graph TD; A[User uploads CSV] --> B[Dataset Understanding Agent]; B --> C[EDA Agent]; C --> D[Preprocessing Agent]; D --> E[Model Selection Agent]; E --> F[Evaluation Agent]; F --> G[Report Agent]; G --> H[Deployment Agent]; H --> I[Final output: Best model, metrics, report, saved model, FastAPI prediction endpoint];
```

Module Purpose

1. `main.py`

This is the FastAPI entry point.

Purpose:

```
Accept dataset upload
Receive target column
Trigger AutoML pipeline
Return result to frontend
Expose prediction endpoint later
```

It should not contain ML logic.

2. `dataset_agent.py`

This agent understands the dataset.

Purpose:

```
Read CSV
Check rows and columns
Identify data types
Detect missing values
Detect duplicate rows
Identify numerical and categorical columns
Validate target column
Detect classification or regression task
```

Output example:

```
{
  "rows":7043,
  "columns":21,
  "target_column":"Churn",
  "task_type":"classification",
  "missing_values": {...},
  "numeric_features": [...],
```

```
"categorical_features": [...]  
}
```

3. `eda_agent.py`

This agent performs automatic EDA.

Purpose:

```
Generate summary statistics  
Check class distribution  
Detect imbalance  
Find correlations  
Detect outliers  
Generate useful charts
```

Output:

```
EDA summary  
Class balance report  
Correlation insights  
Outlier report  
Saved plots
```

4. `preprocessing_agent.py`

This agent prepares data for ML models.

Purpose:

```
Separate X and y  
Handle missing numerical values  
Handle missing categorical values  
Encode categorical columns  
Scale numerical columns  
Split train/test data  
Create reusable preprocessing pipeline
```

Production goal:

```
Save preprocessor.pkl  
Reuse same preprocessing during prediction
```

This is very important for deployment.

5. `model_agent.py`

This agent trains multiple baseline models.

Purpose:

```
Train Logistic Regression  
Train Decision Tree  
Train Random Forest  
Train Gradient Boosting  
Train KNN  
Train SVM  
Train XGBoost later
```

For MVP, start with:

```
Logistic Regression  
Random Forest  
Decision Tree  
Gradient Boosting
```

6. `evaluation_agent.py`

This agent compares models.

Purpose:

```
Evaluate classification models  
Evaluate regression models  
Compare metrics  
Select best model  
Generate confusion matrix  
Store results
```

Classification metrics:

```
Accuracy  
Precision  
Recall  
F1-score  
ROC-AUC
```

Regression metrics:

```
MAE  
MSE  
RMSE  
R2 Score
```

7. `tuning_agent.py`

This agent improves the best model.

Purpose:

```
Tune hyperparameters  
Use GridSearchCV first  
Use RandomizedSearchCV or Optuna later  
Return improved model
```

MVP version:

```
Skip heavy tuning first  
Add RandomizedSearchCV after baseline pipeline works
```

8. `report_agent.py`

This agent generates the final report.

Purpose:

```
Explain dataset summary
Explain task type
Show preprocessing steps
Show model comparison
Show best model
Show metrics
Show limitations
Suggest next steps
```

MVP output:

```
HTML report
```

Later:

```
PDF report
```

9. `deployment_agent.py`

This agent prepares deployment files.

Purpose:

```
Save best_model.pkl
Save preprocessor.pkl
Create FastAPI /predict route structure
Create sample input JSON
Create README deployment instructions
```

Final production output:

```
model.pkl
preprocessor.pkl
prediction_schema.json
FastAPI endpoint
Dockerfile
```

10. `automl_pipeline.py`

This is the orchestrator.

Purpose:

```
Call each agent in correct order
Pass outputs between modules
Handle errors
Return final result
```

Flow:

```
dataset_report=DatasetAgent.analyze()
eda_report=EDAAGent.run()
processed_data=PreprocessingAgent.prepare()
model_results=ModelAgent.train()
evaluation=EvaluationAgent.evaluate()
report=ReportAgent.generate()
deployment=DeploymentAgent.prepare()
```

This file is the heart of the project.

MVP Scope We Should Build First

For the first version, build only this:

```
CSV upload
Target column input
Dataset analysis
Task detection
Preprocessing
Train 4 models
Evaluate models
Select best model
Save model
Return metrics
```

Do not add LangGraph, MLflow, ChromaDB, Optuna, or Docker immediately.

First make the ML engine work.

```
ml_engineer_agent/
|
|— backend/
|   |— agents/
|   |   |— dataset_agent.py
|   |   |— eda_agent.py
|   |   |— preprocessing_agent.py
|   |   |— model_agent.py
|   |   |— tuning_agent.py
|   |   |— evaluation_agent.py
|   |   |— report_agent.py
|   |   |— deployment_agent.py
|   |
|   |— pipelines/
|   |   |— automl_pipeline.py
|   |
|   |— utils/
|   |   |— helpers.py
|   |   |— metrics.py
|   |   |— plots.py
|   |   |— logger.py
|   |
|   |— models/
|   |— reports/
|   |— uploads/
|   |— mlruns/
|   |
|   |— config.py
|   |— main.py
|   |— requirements.txt
|   |
|— frontend/
|   |— streamlit_app.py
|   |
|— notebooks/
|— datasets/
```

- |
- |— Dockerfile
- |— docker-compose.yml
- |— .gitignore
- |— README.md